Senior Project

Kevin Rutkowski

Spring 2009

Table of Contents

Introduction and Purpose

The Goal My Senior Project was to make a program (written in C++) that plays "Guitar hero" and "Rockband" like games for the Xbox 360. The challenges I faced were: how to play the games guitar, how to write to ports on the computer, how to get and use the game files for what I needed, and algorithms for playing the actual songs.

The first problem I faced was how to control the game's guitar. The first thing I tried was to use the Lego Mind storm collection which is basically Lego's that you can use to build and program motorized robots. This method didn't work out because no matter the configuration I tried, their movement was to slow for the reaction times that I needed which were generally in the milliseconds. I did learn a lot of the program language (NQC) and how to program the Mind storm before I found out that it wasn't good enough for what I needed. The second method I tried was a parallel port relay board (http://www.electronickits.com/kit/complete/elec/ck1601.htm) I bought online. This relay board contained 8 relays which is basically a switch that is turned on and off via hexadecimal outputs sent from the computer. At first I was going to use electric solenoids to push the buttons on the guitar, but then I realized it would be much more efficient to just run wires from the guitars buttons directly to the relay board. This worked for all functions of the guitar but the whammy bar, which uses a different method than a normal switch which makes it incompatible with the relay board. The only effect this has on the game is the amount of star power it will generate on the sustained star power notes, so it isn't a must have.

After the problem of how to play the guitar was solved, my next goal was learning how to write to the parallel port on the computer. The biggest problem was getting passed operating system security. Windows operating systems from Windows NT and forward do not allow direct access to ports for security reasons. There are a few third party libraries out there that will allow your program to run in Ring 0 which is kernel mode verse the user mode ring that has restricted access. The kernel is the main component of any operating system. Generally its responsibilities are managing system resources which include I/O ports which is what was needed. The easiest method for doing this was a program called UserPort (http://www.ett.co.th/download/UserPort.pdf). All you need to do in this program is give it a port range and it opens it up for the "user mode" portion of the operating system to use. Another application that easily allows this is

called porttalk22 (http://drivers.softpedia.com/progDownload/PortTalk-Windows-NT-I-O-Port-Device-Driver-22-Download-18278.html). With porttalk22 you can either include it in your code like a third party library, or you can run your application from the command line and it will give you temporary access to whatever port you specify, refer to the documentation for more information. I chose UserPort because the computer that I used wasn't connecting to any network so there was no outside security risk.

The next problem faced was to get and use the game files. The Xbox 360 versions of Rockband and Guitar Hero are heavily encrypted so I was unable to get the files from there. After some further research, the Playstation 2 games were not encrypted and can easily be read from any computer with a DVD player. The next step was to extract MAIN_0.ark from the game. This file contains all the information for the songs included such as name, drum charts, guitar charts, and many other things. After it was extracted from there are programs out there that go through this and pick out exactly what you need. The program I used is called extr25 (http://www.volny.cz/nova-software/files/extr25.zip) and you tell it which type of files you wish to extract, in my case I told it to look for ".mid" or ".midi". Once I got the actual midi files I needed to decode those and make them into human readable format so I could then start to decipher those. The program I used to do that is FeedBack0.97b (http://www.scorehero.com/forum/viewtopic.php?t=73070&start=0&postdays=0&postorder=asc&highlight=). After Feedback was downloaded, you give it a midi file and it creates a chart file out of it which was what I needed so I can read it. The chart file contained all the information needed to make the program work, such as the notes to play and delay between the notes. There is also a Perl script included to parse this file and create a file that is readable by the main program.

The final challenge was to write the actual program that plays the song. The basic algorithm I used to play songs with my program was: open a file containing all the information I needed, read it into a queue, and then pop off the front of the queue delaying appropriately. There was more functionality to the program then just playing songs, but the above algorithm was what was most important.

The timing is the biggest issues within the program itself. The delay between notes is handled by the QueryPerformanceCounter() WIN API functions which can accurately determine time lengths. While the time works for some songs, it doesn't work as well for others.

During this project I was constantly moving files between computers because the computer I mainly worked on it didn't have a parallel port so I created a series of batch files that made this easier for me which I've also included in this binder, all work under the assumption that there is an environmental variable named "project" that leads to where the program files are located.

How to Use

1) Initially you need the midi file of the song you want to play. To get the midi file you need to get it off the game disc using the ext25 program mentioned earlier.
2) Place your midi file in the "Songs" directory of Feedback.
3) Open up Feedback and navigate to the midi file from the game you want to use.
4) Save that as a chart file.
5) After you have the chart file, run the Perl script on the chart file. This creates a file named the same as the chart file, but in a ".dat" format in the "songs" directory.
6) Place the ".dat" file in the same directory as the main program.
7) Run the main program and choose the function you wish to use, enter your file name and wait for the song to start.
8) Once the song is playing it is up to the user to hit the Enter or Space Key once the first note passes. This starts the program playing the song. Due to user input there is a chance that the delays will be off during the song.

Once you have the ".dat"songs in the directory you can skip steps 1 through 5, but if you want new songs you will have to do all the steps.

I have also written some batch files that make moving the files from place to place a lot easier. In order to use them you need to create an environmental variable in windows named "project" that leads to the directly the directory that the program is in.

Code and Scripts

Main.cpp
This file contains the interface and all the menu animation options.

A description of all the functions within main.cpp:

- Main-
  - calls init and handles all of the user's input (directional, enter, and escape keys) and calls selector to handle animations
- Init-
  - sets up the user interface
- Selector-
  - highlights whichever option the user is on while storing the current selection, recursively deselecting previous
- intToString-
  - simple function that converts an integer into a string for easy output
- play-
  - calls methods from relay class with the stored current selection

Relays.h

This file contains all the methods required to manipulate the relay board itself. It contains fixed output methods, custom output methods, and the methods used to play the songs either manually or automatically.

Description of all functions within Relays.h:

- Constructor:
  - main.cpp creates a "Relays" object passing it a hexadecimal value to which port the program will write to (in this case 0x378), pointer to the console class (handles all of the printing to screen), a Boolean value noPortDebug for when I was working on a computer that did not have parallel port (true for no port, false if there is a port that you can write to), and finally max selection value. The max selection value is just used as a reference to where the program should print messages to the screen because every time I added new functionality to the program I didn't want to have to go back and input new values for everything.
- Fixed output methods region:
  - Green
    - Sent the hex-value for the function named fret to the guitar or printed it to screen depending on debug mode
  - Red
    - Sent the hex-value for the function named fret to the guitar or printed it to screen depending on debug mode
  - Yellow
    - Sent the hex-value for the function named fret to the guitar or printed it to screen depending on debug mode
  - Blue
    - Sent the hex-value for the function named fret to the guitar or printed it to screen depending on debug mode
  - Orange
    - Sent the hex-value for the function named fret to the guitar or printed it to screen depending on debug mode
  - Up
    - Sent the hex-value for the function named fret to the guitar or printed it to screen depending on debug mode
  - Down

        - Sent the hex-value for the function named fret to the guitar or printed it to screen depending on debug mode
    - Reset
        - Sent 0x0 to the board to reset all relays to off
    - gameMode
        - A method used to control in game actions so the user can control the game from the computer rather than the guitar.
- Custom output method region
    - Custom
        - Method used to ask user for hexadecimal input to output to the board.
- Play song auto region
    - Playsongauto
        - Method that asks user to input the name of the ".dat" file that will be used to play the song. Initially it calls gamemode so the user can navigate to the song they want, start it up then type in the name. When the first note goes by the user has to hit a key in order to start playing the song. User can end the song at anytime by pressing escape key.
- Play song man region
    - Playsongman
        - Method that asks user to input the name of the ".dat" file that will be used to play the song. Initially it calls gamemode so the user can navigate to the song they want, start it up then type in the name. In this mode the user can tap along to the song using the keyboard. This method also measure the time in between each press and records it to a file for future reference. User can end song at any time by pressing the escape key.
- Private methods and variables
    - Openfile
        - Opens the specified file or returns false if not found. This method reads the file calls load queue method to load the song queue for playing.
    - Loadqueue
        - Method that takes hex value to send to the relay board and a delay for the next note and stores it in the song queue.

Timer.h

This class is used to measure time differences, and a customized sleep function that more accurately waits milliseconds then the windows API Sleep() method.

Chart2dat.pl

This script asks the user for a file name. Once it verifies the file exists, it then parses the file looking for the resolution, beats per minute as they are related to each "tick" in the song and each "tick as it relates to what frets have to be played at that current "tick".

More explanation on the chart files:

Every song in the game has a different chart file. The resolution found inside the chart file is what is used to convert the "ticks" into milliseconds since the "ticks" aren't just finite times. The very first section of the chart files is called sync track. Inside this section there are ticks on the left, and bpms on the right. For example a line looks like:

3840 = B 140900

Where 3840 means when the song "ticks" are 3840 or greater, the bpm * 1000 is 140,900 or 140.9 bpm. These values are included in the formula that is used to convert the ticks to milliseconds.

The next section of this file that my script looks for is called expertsingle. Expert single is the expert guitar chart. It is formatted like:

4080 = N 3 0

For example, at tick 4080 the note to be played is 3 where 0=green, 1=red, 2=yellow, 3=blue, 4=orange. If there are two lines where the "ticks" are equal such as:

4080 = N 3 0

4080 = N 4 0

Then at timestamp 4080 there is a chord that will have to be played consisting of blue and orange. The formula I used to convert ticks to ms is tickdiff/resolution * 60/bpm.

Hardware

Xbox 360

The Xbox 360 is used to play the Rockband game.

Rockband Guitar

This is the controller for the Rockband game. There are 5 frets, green, red, yellow, blue, orange and there is also a strum bar, d-pad, start select and guide button. My program utilizes all frets, d-pad (mimics strum bar), and the select button which activates overdrive.

Relay Board

The relay board purchased from http://www.electronickits.com/kit/complete/elec/ck1601.htm is a pretty simple device. You plug it into your computers parallel port and it is activated by hexadecimal values sent to it.

There are a few ways you can wire it. Each really has three ports, normally open (NO), normally closed (NC), and Current (C). NO is when the relay is normally open, meaning the circuits are usually disconnected. This is the one I used because I wanted to turn the frets on the guitar on when I wanted them to be, not just all the time. You plug the ground into the normally open, and the hot wire into the Current slot. Normally closed would mean the circuit is closed and when you activate the relay it will open to circuit, turn it off. To use the NC slot you put your ground into NC and hotwire into Current.

There are 8 relays so you can control power to 8 different objects. In this case I opened up the Rockband guitar and attached leads to each of the frets and their grounds on that circuit board so I can control when they are switched "on". Each relay is controlled by a hexadecimal value represented by their 8 bit binary counterpart. For example, 0x1 == 0000 0001 turns on relay 1 and 0x2 == 0000 0010 turns just relay two on. If I wanted to turn both 1 and 2 on I would add them together 0x1+0x2 =0x3 == 0000 0011. Therefore the biggest value you can send to the parallel port is 0xFF == 1111 1111 and the smallest is 0x0 == 0000 0000.

```
#include <conio.h>
#include <windows.h>
#include <string>
#include "Relays.h"
#include "Console.h"
using namespace std;

#pragma region declarations
//global variables
Console *con = new Console(75,50);//width, length
const int maxSelection=13;// how many selections there are in menu
Relays r(0x378, con, false, maxSelection);// LPT port to write to, pointer to console, and
true for no port debug mode/ false for otherwise, maxSelection for debug

//enumerations
enum DIR {UP,DOWN,NONE};// determine which way the user came from in the menu for
deselecting

//function headers
void selector(int,DIR,bool=false);//menu animation
void init();//initialize menu
string intToString(int);//convert int to string
void play(int); //action once a menu item is selected passing the selected items index
#pragma endregion

int main()
{
        init();
        char input=NULL;
        int selection =1;

        while(input != 27)
        {
                input=_getch();//gets user input
                switch(input)
                {
                case 'H'://up
                        selection-=1;
                        if(selection ==0)
                                selection=maxSelection;
                        selector(selection,UP);
                        break;
                case 'P'://down
```

```
                        selection+=1;
                        if(selection == maxSelection+1)
                                selection=1;
                        selector(selection,DOWN);
                        break;
                case '\n':
                case '\r':
                        play(selection);
                        con->printStringClearLine(0,maxSelection+3,"");//clear
any error messages
                        break;
                default:
                        break;
                }
        }
        return 0;
}
//initializes screen
void init()
{
        con->printString(0,0,"Relay Board Interface- ESC key to exit.");


        for (int i=maxSelection; i>1;i--)
        {
                selector(i,NONE,true);//print out menu
        }
        selector(1,NONE,false);//highlight the first selection
}

//for Menu animation
void selector(int sel, DIR dir, bool deselector)
{
        if(deselector)
                con->setColor(white,black);//white words, black background
        else
                con->setColor(black,white);//black words, white background

switch(sel)
{
case 1: con->printStringClearLine(0,1,"1. Green");
        break;
case 2: con->printStringClearLine(0,2,"2. Red");
        break;
```

```
case 3: con->printStringClearLine(0,3,"3. Yellow");
        break;
case 4: con->printStringClearLine(0,4,"4. Blue");
        break;
case 5: con->printStringClearLine(0,5,"5. Orange");
        break;
case 6: con->printStringClearLine(0,6,"6. Up");
        break;
case 7: con->printStringClearLine(0,7,"7. Down");
        break;
case 8: con->printStringClearLine(0,8,"8. Back");
        break;
case 9: con->printStringClearLine(0,9,"9. Custom");
        break;
case 10: con->printStringClearLine(0,10,"10. Game Mode");
        break;
case 11: con->printStringClearLine(0,11,"11. Reset");
        break;
case 12: con->printStringClearLine(0,12,"12. Play Song Automatically");
        break;
case 13: con->printStringClearLine(0,13,"13. Play Song Manually");
}
        con->setColor(white,black);//reset colors to white word black background


        //determine if you finished selecting something so you can recursively de
select it
        if(!deselector)
        {
                switch(dir)
                {
                case UP:
                        sel+=1;
                        if(sel == maxSelection+1)
                                sel=1;
                        selector(sel,NONE, true);// if i got here from pressing up
deselect below me
                        break;


                case DOWN:
                        sel-=1;
                        if(sel == 0)
                                sel=maxSelection;
                        selector(sel,NONE, true);//if i got here from pressing
```

```
down deleselect above me
                                        break;
                                default:
                                        break;
                                }
                        }
}

//activate option specified by menu
void play(int sel)
{
                switch(sel)
                {
                case 1:
                                r.green();
                                break;
                case 2:
                                r.red();
                                break;
                case 3:
                                r.yellow();
                                break;
                case 4:
                                r.blue();
                                break;
                case 5:
                                r.orange();
                                break;
                case 6:
                                r.up();
                                break;
                case 7:
                                r.down();
                                break;
                case 8:
                                r.back();
                                break;
                case 9:
                                r.custom();
                                break;
                case 10:
                                r.gameMode();
                                break;
```

```
		case 11:
			r.reset();
			break;
		case 12:
			r.playSong();
			break;
		case 13:
			r.playSongMan();
			break;
		default:
			con->printStringClearLine(0,maxSelection+2, "Error in Selection");
			break;
		}
		//con->printStringClearLine(0,maxSelection+1,"Selection " + intToString(sel)
+ " Pressed."); //debug code
}
//convert ints to strings
string intToString(int num)
{
		char convert[10];
		sprintf_s(convert,"%d",num);


		return convert;
}
```

```
#pragma once
#include <string>
#include <conio.h>
#include <windows.h>

enum Color {black=0, blue=FOREGROUND_BLUE, green=FOREGROUND_GREEN,
red=FOREGROUND_RED,
purple=FOREGROUND_BLUE + FOREGROUND_RED,
cyan=FOREGROUND_BLUE+FOREGROUND_GREEN,
brown=FOREGROUND_GREEN+FOREGROUND_RED,
white=FOREGROUND_BLUE+FOREGROUND_GREEN+FOREGROUND_RED};

void waitForEnter();// wait until the user presses ENTER
bool getInput(char& ch);

class Console {
public:
   Console(int width, int height);
            void setColor(Color fore, Color back) {SetConsoleTextAttribute(m_hConsole,
fore + (back<<4) );}
            void waitForEnter();// wait until the user presses ENTER
   void gotoXY(int x, int y); //moves cursor to position x (from left), y (from top)
             //0,0 is upper left corner
            void printChar(int x, int y, char ch); //prints ch at x,y, moves cursor over 1 to
right
            void printChar(char ch) {_putch(ch);} //prints ch where current cursor is,
moves cursor over 1 to right
   void printString(int x, int y, const std::string& s);
   void printString(const std::string& s);
   void printStringClearLine(const std::string& s); //clears rest of line after string printed
   void printStringClearLine(int x, int y, const std::string& s);
            void printInt(int x, int y, int val); //print an integer value at x,y
            void printInt(int val); //print an integer value
            void delay(int amount) {Sleep(amount);}
private:
   HANDLE  m_hConsole;
   int     m_width;
   int     m_height;
};
```

```
#include "Console.h"

void waitForEnter() {// wait until the user presses ENTER
 int ch;
 do {
            ch = _getch();
 } while ( ch != '\n'  &&  ch != '\r');
}

/* The getInput function checks whether the user has hit a key and immediately
 returns (so it does NOT wait for a key to be pressed).  It returns:
   true if the user has hit a key; in this case, the ch variable is set to
        the typed character
   false if the user has not hit any key; in this case, the ch variable
        will be unchanged*/
bool getInput(char& ch) {
 if (_kbhit()) {//has key been hit?
   int c = _getch();


            if (c != 0xE0)  // first of the two sent by arrow keys, E0 is escape
     ch = c;
   else {
    c = _getch(); //get second char sent by escape sequence from arrows
    switch (c) {
     case 'K':  /* left  */ ch = '4'; break;
     case 'M':  /* right */ ch = '6'; break;
     case 'H':  /* up    */ ch = '8'; break;
     case 'P':  /* down  */ ch = '2'; break;
     default:            ch = '?'; break;//special character but not arrow
    }
   }
   return true;
 }
 return false;
}


Console::Console(int width, int height) : m_width(width), m_height(height) {
 m_hConsole = GetStdHandle(STD_OUTPUT_HANDLE);
 CONSOLE_CURSOR_INFO x;
 GetConsoleCursorInfo(m_hConsole, &x);
 x.bVisible=false;
 SetConsoleCursorInfo(m_hConsole, &x);//hides the cursor
}
```

```
void Console::gotoXY(int x, int y) {//moves cursor to position x (from left), y (from top)
//0,0 is upper left corner
  if (x < 0  ||  x >= m_width  ||  y < 0  ||  y >= m_height)//invalid x,y
     return;
  COORD pos = { x, y };//same as pos.X=x; pos.Y=y;
  SetConsoleCursorPosition(m_hConsole, pos);
}

void Console::printChar(int x, int y, char ch) {//prints ch at x,y, moves cursor over 1 to
right
  gotoXY(x,y);
  printChar(ch);
}

void Console::printString(int x, int y, const std::string& s) {
  gotoXY(x,y);
  printString(s);
}

void Console::printString(const std::string& s) {
  for (size_t i = 0; i < s.length(); i++)
     printChar(s[i]);
}

void Console::printStringClearLine(int x, int y, const std::string& s) {//clears rest of line
after string printed
  gotoXY(x,y);
  printStringClearLine(s);
}

void Console::printStringClearLine(const std::string& s) {//clears rest of line after string
printed
  printString(s);
  CONSOLE_SCREEN_BUFFER_INFO info;
  GetConsoleScreenBufferInfo(m_hConsole,&info);
  for (int col=info.dwCursorPosition.X; col < m_width; col++)
                printChar(' ');//print spaces for rest of line, console b/c don't know if we
started at 0
}

void Console::printInt(int x, int y, int val) {//print an integer value
  gotoXY(x,y);
```

```
  printInt(val);
}
void Console::printInt(int val) {//print an integer value
  char buf[12]={0};
  _itoa_s(val, buf, 11, 10);//safely convert int to ascii text
  printString(buf);  //constructor for string automatically called for char *
}
```

```
#ifndef TIMER_H
#define TIMER_H
#include <sys/timeb.h>
#include <windows.h>
#include <conio.h>
#include <ctime>

class timer{
public:
	void sleepMilliseconds(double delay)
	{
		/*clock_t endwait;
		endwait = clock () + delay * CLOCKS_PER_SEC/1000 ;
		while (clock() < endwait) {}*/


		__int64 current, end, freq;

		QueryPerformanceFrequency((LARGE_INTEGER*) &freq);
		QueryPerformanceCounter((LARGE_INTEGER*) &current);
		QueryPerformanceCounter((LARGE_INTEGER*) &end);

		unsigned int currentTime=(unsigned int)((current *1000 / freq) &
0xffffffff);
		double endTime= ((end * 1000 / freq) & 0xffffffff) + delay;
		while(currentTime < endTime)
		{
			QueryPerformanceCounter((LARGE_INTEGER*)
&current);
			currentTime=(unsigned int)((current *1000 / freq) &
0xffffffff);
		}
	}

	double GetMilliCount()
	{
		// Something like GetTickCount but portable
		// It rolls over every ~ 12.1 days (0x100000/24/60/60)
		// Use GetMilliSpan to correct for rollover
		timeb tb;
		ftime( &tb );
		double nCount = double(tb.millitm + (tb.time & 0xfffff) * 1000);
		return nCount;
```

```
	}

	double GetMilliSpan( double nTimeStart )
	{
		double nSpan = GetMilliCount() - nTimeStart;
		if ( nSpan < 0 )
			nSpan += 0x100000 * 1000;
		return nSpan;
	}
	//int main()
	//{
	//		int nTimeStart = GetMilliCount();
	//// ... processing
	////		_getch();
	//		Sleep(2000);
	//int nTimeElapsed = GetMilliSpan( nTimeStart );
	//cout<<nTimeElapsed<<endl;
	//
	//		return 0;
	//}
};
#endif
```

```
#ifndef Relays_H
#define Relays_H
#include <string>
#include <iostream>
#include <fstream>
#include <queue>
#include <list>
#include "Console.h"
#include "timer.h"
using namespace std;

class Relays{
public:
	//set variables
#pragma region Constructor/Destructor
	Relays(int port, Console *c, bool noportdebug, int maxSelection)
	{
		PORT=port;//address of LPT  to be used
		con = c;//pointer to console for input/output
		noPortDebug=noportdebug;//determine if in debug mode
		stringLine=maxSelection+2;//line to print strings to
	}
	~Relays()
	{
		reset();
	}
#pragma endregion
	//methods with fixed outputs
#pragma region fixedOutputMethods
	void green()
	{
		if(!noPortDebug)
			_outp(PORT, GREEN);
		else
			con->printStringClearLine(0,stringLine,"Green");//debug
code
	}
	void red()
	{
		if(!noPortDebug)
			_outp(PORT, RED);
		else
			con->printStringClearLine(0,stringLine,"Red");//debug
```

```
code
        }
        void yellow()
        {
                if(!noPortDebug)
                        _outp(PORT, YELLOW);
                else
                        con->printStringClearLine(0,stringLine,"Yellow");//debug
code
        }
        void blue()
        {
                if(!noPortDebug)
                        _outp(PORT, BLUE);
                else
                        con->printStringClearLine(0,stringLine,"Blue");//debug
code
        }
        void orange()
        {
                if(!noPortDebug)
                        _outp(PORT,ORANGE);
                else


con->printStringClearLine(0,stringLine,"Orange");//debug code
        }
        void up()
        {
                if(!noPortDebug)
                        _outp(PORT,UP);
                else
                        con->printStringClearLine(0,stringLine,"Up");//debug
code
        }
        void down()
        {
                if(!noPortDebug)
                        _outp(PORT,DOWN);
                else
                        con->printStringClearLine(0,stringLine,"Down");//debug
code
        }
        void back()
```

```
	{
		if(!noPortDebug)
			_outp(PORT,BACK);
		else
			con->printStringClearLine(0,stringLine,"Back");//debug
code
	}
	void reset()
	{
		if(!noPortDebug)
			_outp(PORT, 0x0);
		else
			con->printStringClearLine(0,stringLine,"Reset");//debug
code
	}
	void gameMode()
	{
		char input=NULL;
		con->printStringClearLine(0,stringLine+1,"up arrow-UP");
		con->printStringClearLine(0,stringLine+2,"down arrow-DOWN");
		con->printStringClearLine(0,stringLine+3,"enter-GREEN");
		con->printStringClearLine(0,stringLine+4,"backspace-RED");
		con->printStringClearLine(0,stringLine+5,"F-see available files");
		con->printStringClearLine(0,stringLine+6,"q-QUIT");



		while(input != 'q' && input != 27)
		{
			input=_getch();//gets user input
			switch(input)
			{
			case 'f':
				system("start dirs");
				continue;
			case 'H'://up
				up();
				break;
			case 'P'://down
				down();
				break;
			case '\n'://return
			case '\r'://return
				green();
```

```
                                        Relays
                                        break;
                                case 8://backspace
                                        red();
                                        break;
                                default:
                                        break;
                                }
                                Sleep(50);
                                reset();
                        }
                        for(int i= stringLine+1; i<=stringLine+6;i++)
                                con->printStringClearLine(0,i,"");
        }
#pragma endregion
        //asks user for hex digit and outputs it
#pragma region customOutputMethod
        void custom()
        {
                string hexa;
                int hexadecimal=-1;
                con->printString(0,stringLine,"input hex digit to output: ex(0x99):
");
                cin>>hexa;


                sscanf_s(hexa.c_str(),"%x",&hexadecimal);
                //checks if digit entered is in hex
                if(hexadecimal != -1 && (hexa[0] == '0' && (hexa[1] =='x' ||
hexa[1]=='X')))
                {
                        //checks if hex digit is in range
                        if(hexadecimal >=0 && hexadecimal <=MAX)
                        {
                                if(!noPortDebug)
                                        _outp(PORT, hexadecimal);

                                //convert from hex to string
                                char port[10];
                                sprintf_s(port,"%x",PORT);


con->printStringClearLine(0,stringLine,"Writing "+hexa+" to port "+port);
                        }
                        else
```

```
                                        Relays
                                         con->printStringClearLine(0,stringLine,"hex
digit must be from 0x0 to 0xff");
                        }
                        else
                                con->printStringClearLine(0,stringLine,"Not a hex Digit.
Must start with 0x");
        }
#pragma endregion
        //asks user for file and plays it automatically
#pragma region PlaySongAuto
        void playSong()
        {
                if(!openFile())//if file fails to open exit
                        return;
                char input=NULL;//input char used to exit song thats playing
                con->printStringClearLine(0,stringLine,"Please navigate to the
song.");
                gameMode();
                con->printStringClearLine(0,stringLine,"Press any key to play
song");
                _getch();
                con->printStringClearLine(0,stringLine,"Playing Song...");
                timer clock;
                double del=.5;
                while(!song.empty())
                {
                        //ability for user to cancel the song playing
                        getInput(input);


                        switch(input)
                        {
                                case '8':
                                        del+=.1;
                                        break;
                                case '2':
                                        del-=.1;
                                        break;
                                case 27:
                                        reset();

con->printStringClearLine(0,stringLine,"Song Interupted");
                                        while(!song.empty())
                                                song.pop();//empty
```

```
queue so if another song is to be played there will be no left overs
                                                return;
                                                break;
                                        default:
                                                break;

                                }
                                if(!noPortDebug)
                                {
                                        _outp(PORT, song.front().hex);// turn relays
on
                                }
                                else
                                {
                                        //debug code
                                        char convert[10];
                                        sprintf_s(convert,"%x",song.front().hex);
                                        con->printStringClearLine(0,stringLine+1,
convert);
                                }
                                clock.sleepMilliseconds(song.front().delay-del);
                                song.pop();//dequeue current chart
                        }
                        con->printStringClearLine(0,stringLine,"Song Over");//let the user
know the song is over
                        _getch();
                        reset();//turn all relays off
                }
#pragma endregion
                //user plays song by tapping a key, delay is written to a file delays.txt
#pragma region playSongMan
                void playSongMan()
                {
                        if(!openFile())//if file fails to open exit
                                return;


                        gameMode();
                        char input=NULL;//input char used to exit song thats playing
                        con->printStringClearLine(0,stringLine,"Tap any key to play the
note, Q or esc to quit");
                        ofstream output("DELAYS.txt",ios::out);
                        timer clock;
                        double start=0;
```

```
int finish=0;

while(!song.empty())
{
	//ability for user to cancel the song playing
	input=_getch();
	if(start != 0)
		output << clock.GetMilliSpan(start)<<endl;
	start = clock.GetMilliCount();
	switch(input)
	{
	case 'q':
	case 'Q':
	case 27:
		reset();
		con->printStringClearLine(0,stringLine,"Song
Interupted");
		while(!song.empty())
			song.pop();//empty queue so if
another song is to be played there will be no left overs
		return;
		break;
	default:
		break;
	}
	if(!noPortDebug)
	{
		_outp(PORT, song.front().hex);// turn relays
on
	}
	else
	{
		//debug code
		char convert[10];
		sprintf_s(convert,"%x",song.front().hex);
		con->printStringClearLine(0,stringLine+1,
convert);
	}
	song.pop();//dequeue current chart
}
con->printStringClearLine(0,stringLine,"Song Over");
reset();//turn all relays off
output.close();
```

```
        }
#pragma endregion
private:
#pragma region Variables
        int PORT;//set the default port to send to
        Console *con;//console pointer to write to the screen
        bool noPortDebug;//debug boolean
        int stringLine;//line to print strings to
        struct notesToPlay{//struct that holds what frets to play and the delay till next
note
                char frets[5];
                double delay;
        };
        struct hexValueAndDelay{
                int hex;
                double delay;
        };
        queue<hexValueAndDelay> song;


        static const int        GREEN=0x1,//relay 1
                RED=0x2,//relay 2
                YELLOW=0x4,//relay 3
                BLUE=0x8,// relay 4
                ORANGE=0x10,//relay 5
                UP=0x20,//relay 6
                DOWN=0x40,//relay 7
                BACK=0x80,//relay 8

MAX=GREEN+RED+YELLOW+BLUE+ORANGE+UP+DOWN+BACK;//max value that can
be sent to relay board
#pragma endregion

#pragma region Private Methods
        bool openFile()
        {
                string inputString;
                con->printString(0, stringLine,"Enter file name to open (ex
charlene.dat): ");
                cin>>inputString;
                ifstream inFile(inputString.c_str());//open file

                //check if file is open
                if(!inFile)
```

```
		{
			con->printStringClearLine(0, stringLine,"Incorrect file
name.");
			return false;
		}
		else
		{
			con->printStringClearLine(0,stringLine,"Loading song
chart...");
			notesToPlay notes;
			bool upStrum=true;
			//read in text file to Chart struct and push it into queue
			while(inFile)
			{
				inFile>>notes.delay;//delay
				inFile>>notes.frets[0];//green
				inFile>>notes.frets[1];//red
				inFile>>notes.frets[2];//yellow
				inFile>>notes.frets[3];//blue
				inFile>>notes.frets[4];//orange
				//noteChart.push(notes);
				loadSongQueue(notes, upStrum);
				upStrum=!upStrum;
			}
			//close file
			inFile.close();
			return true;
		}
	}
	void loadSongQueue(notesToPlay noteChart,bool upStrum)
	{
		hexValueAndDelay hexValue;
		hexValue.delay=noteChart.delay;
		hexValue.hex=0;
		if(noteChart.frets[0] != 'N')
			hexValue.hex+=GREEN;// sum up the frets that need to
be pushed
		if(noteChart.frets[1] != 'N')
			hexValue.hex+=RED;// sum up the frets that need to be
pushed
		if(noteChart.frets[2] != 'N')
			hexValue.hex+=YELLOW;// sum up the frets that need to
be pushed
```

```
                if(noteChart.frets[3] != 'N')
                        hexValue.hex+=BLUE;// sum up the frets that need to be
pushed
                if(noteChart.frets[4] != 'N')
                        hexValue.hex+=ORANGE;// sum up the frets that need to
be pushed
                //alternate between up and down strum
                if(upStrum)
                {
                        hexValue.hex+=BACK;
                        hexValue.hex+=DOWN;
                }
                else
                {
                        hexValue.hex+=UP;
                }
                song.push(hexValue);
        }
#pragma endregion
};
#endif
```

```
use Switch;

print "input file to open: ";
$file=<stdin>;
chomp($file);

open(IN, $file) or error();
open(O, ">test.txt");
$file=~ s|(.+)\..+|$1|;
open(OUT,">".$file.".dat");

while (<IN>)
{
	##########################################
	#finds resolution
	##########################################
	if(m|resolution|i)
	{
		($resolution)= m|(\d+)|;
	}

	##########################################
	#stores all synctrac stamps and bpms in array
	##########################################
	if(m|synctrack|i)
	{
		$writeSyncTrack=1;
	}
	if($writeSyncTrack)
	{
		if(m|\d+\s=\sB\s\d+|)
		{
			($tick, $bpm)= m|(\d+)\s=\sB\s(\d+)|;
			push(@syncTrack,$tick);
			push(@bpmArray, $bpm);
		}
		if(m|\}|)
		{
			$writeSyncTrack=0;
		}
	}
	##########################################
	#stores all frets and ticks in array
```

```
		##########################################
		if(m|expertsingle|i)
		{
			$writeChart=1;
		}
		if($writeChart)
		{
			if(m|=\sN|)
			{
				($tick, $fret)= m|(\d+)\s=\sN\s(\d)|;
				push(@ticks, $tick);
				push(@frets, $fret);
			}
			if(m|\}|)
			{
				$writeChart=0;
			}
		}
}
##########################################################################
##
#for loop that creates an array for ticks(without duplicates)		#
#and an array that contains frets to be played with its corresponding tick  #
##########################################################################
##
for ($i=0; $i < $#ticks+1; $i++)
{
			if($ticks[$i] ==$ticks[$i-1])
			{
				$finalFrets[$#finalFrets].=whichFret($frets[$i]);
			}
			else
			{
				push(@finalTicks, $ticks[$i]);
				push(@finalFrets, whichFret($frets[$i]));
			}
}
#extra tick  for the last note
push(@finalTicks, 10000000);
shift(@syncTrack);#remove the initial 0


for($index=0; $index<$#finalTicks+1;$index++)
{
```

```
		if($syncTrack[0] <= $finalTicks[$index+1])
		{
			shift(@syncTrack);
			if($#bpmArray != 0)
			{
				shift(@bpmArray);
			}
		}
		push(@millis, getMilli($finalTicks[$index], $finalTicks[$index+1],
$bpmArray[0]));
}

for($i=0; $i<$#finalFrets+1; $i++)
{
		print OUT $millis[$i].buildFretString($finalFrets[$i]);
}

sub buildFretString{
		$section=shift;
		$element=" ";
		if($section=~ /G/i)
		{

			$element.= "G ";
		}
		else
		{
			$element.= "N ";
		}

		if($section=~/R/i)
		{
			$element.= "R ";
		}
		else
		{
			$element.= "N ";
		}

		if($section=~/Y/i)
		{
			$element.= "Y ";
		}
```

```
		else
		{
			$element.= "N ";
		}

		if($section=~/B/i)
		{
			$element.= "B ";
		}
		else
		{
			$element.= "N ";
		}
		if($section=~/O/i)
		{
			$element.= "O\n";
		}
		else
		{
			$element.= "N\n";
		}
		$element;
}

sub getMilli{
$previoustick=shift;
$currenttick=shift;
$bp=shift;
$bp/=1000;
$difftick=$currenttick-$previoustick;
$output=1000*($difftick/$resolution * 60/$bp);
if($output>1000000)
{
		$output=0;
}
print O $currenttick."\t".$difftick." / ".$resolution." * 60 / ".$bp."\t= ".$output."\n";
$output;
}
sub whichFret{
$parameter=shift;
		switch($parameter)
		{
			case 0 { $f=" G"}
```

```
                case 1 { $f=" R"}
                case 2 { $f=" Y"}
                case 3 { $f=" B"}
                case 4 { $f=" O"}
        }
        $f;
}

sub error{
print "No such file name.\nPress any key to continue.\n";
<stdin>;
die;
}
```

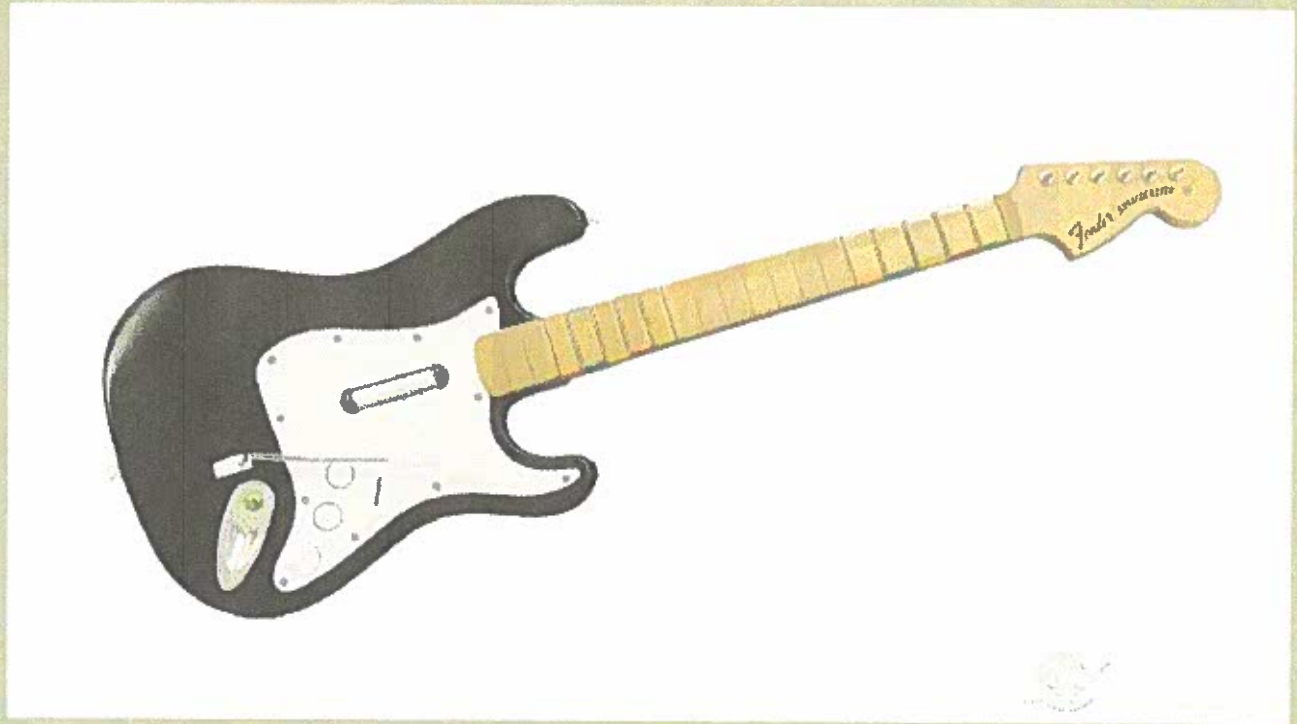
SENIOR PROJECT

Kevin Rutkowski

Spring 2009

Introduction

- Project Description
  - Create a program to play the Rockband game for the Xbox 360



The Game

- How to play the game
  - As a song plays colored frets come down and the player has to hit the combination of the frets and the strum bar they pass the bottom of the screen.

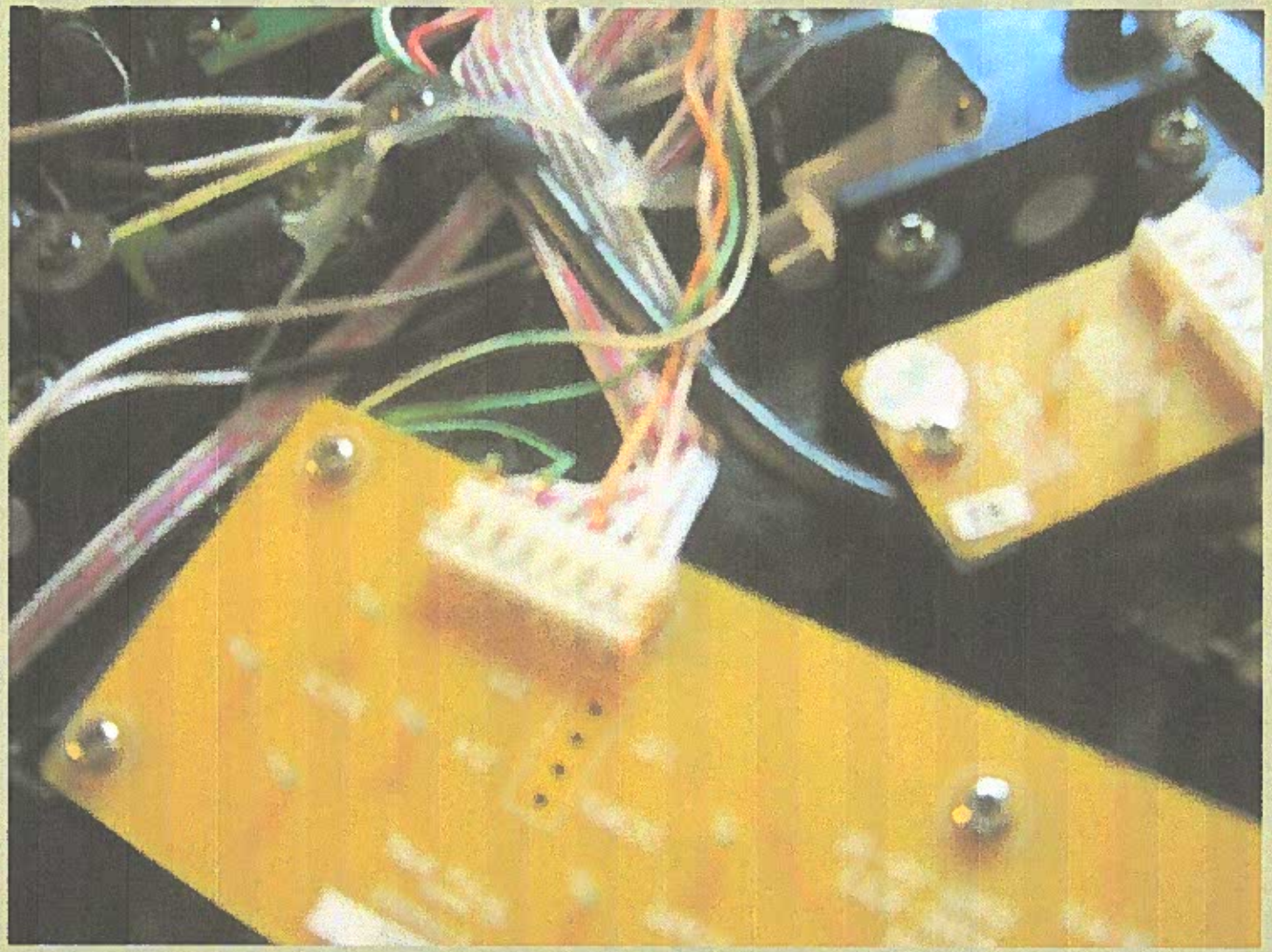
Challenges

- How to play the guitar via computer
- What notes to play and when

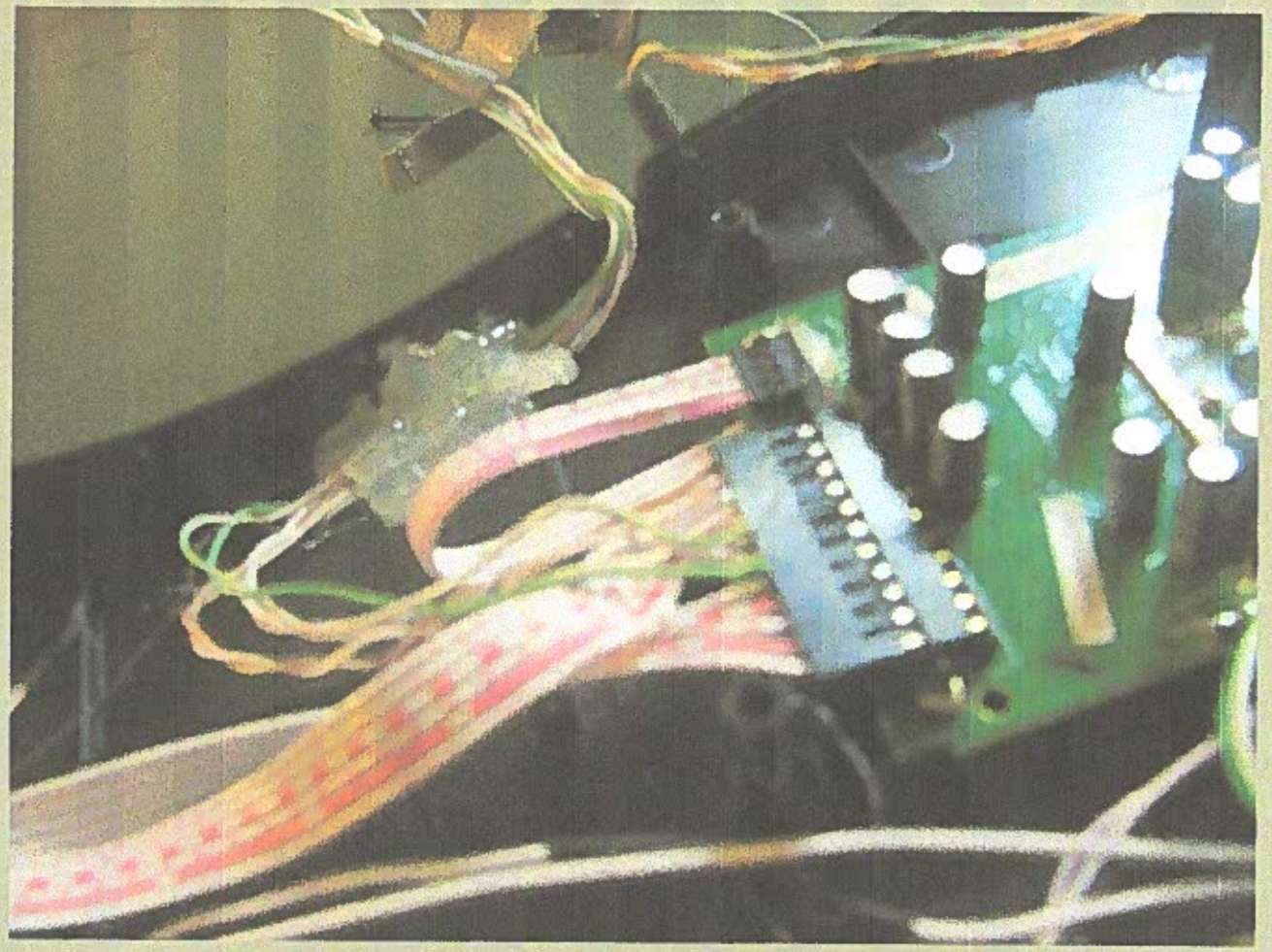
How to play the guitar via computer?

- Methods tried
  - Lego Mind-storm
  - Solenoids & Relay board
- Method used
  - Relay board
    - Parallel port
    - User Port

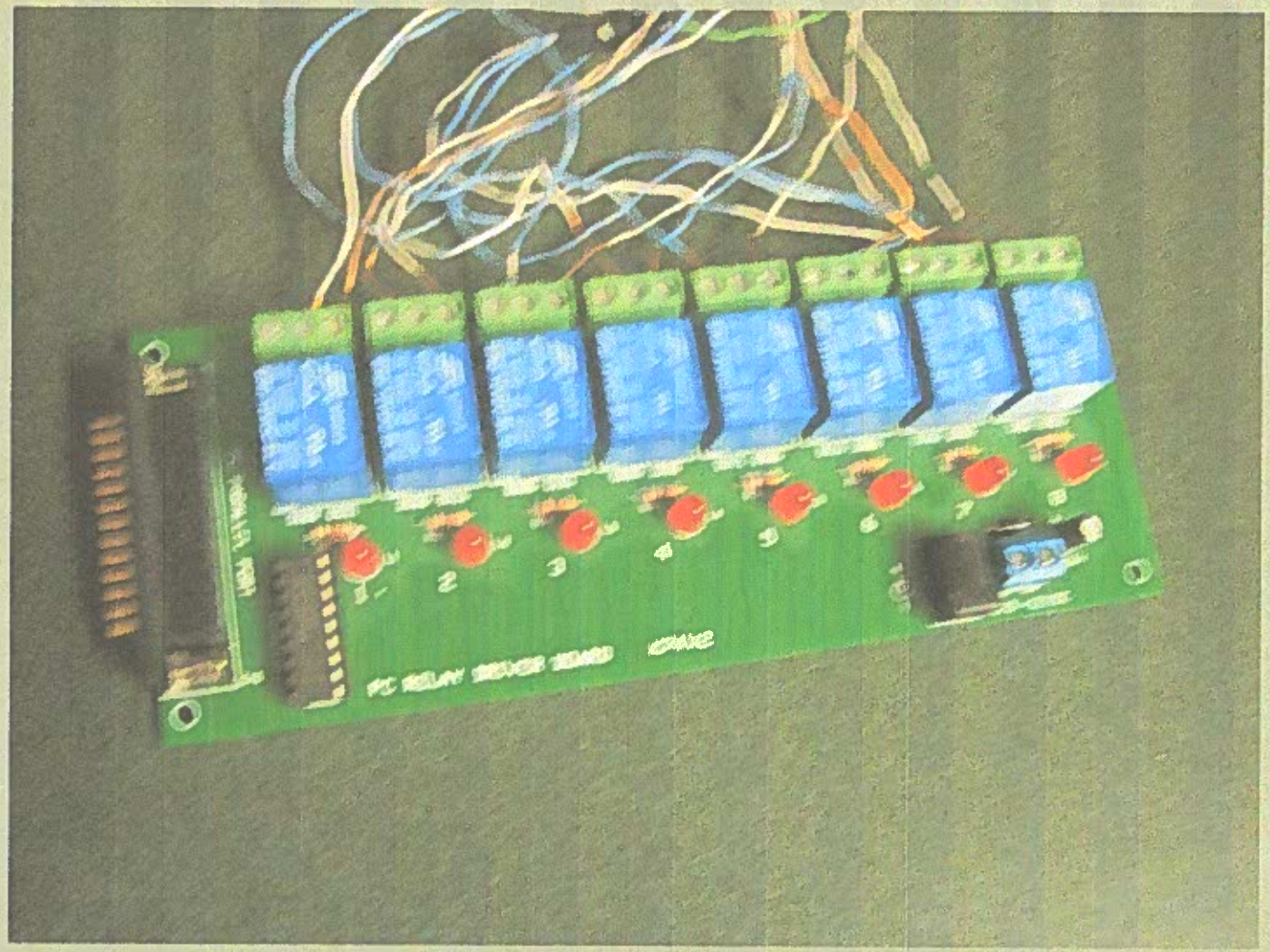
Guitar

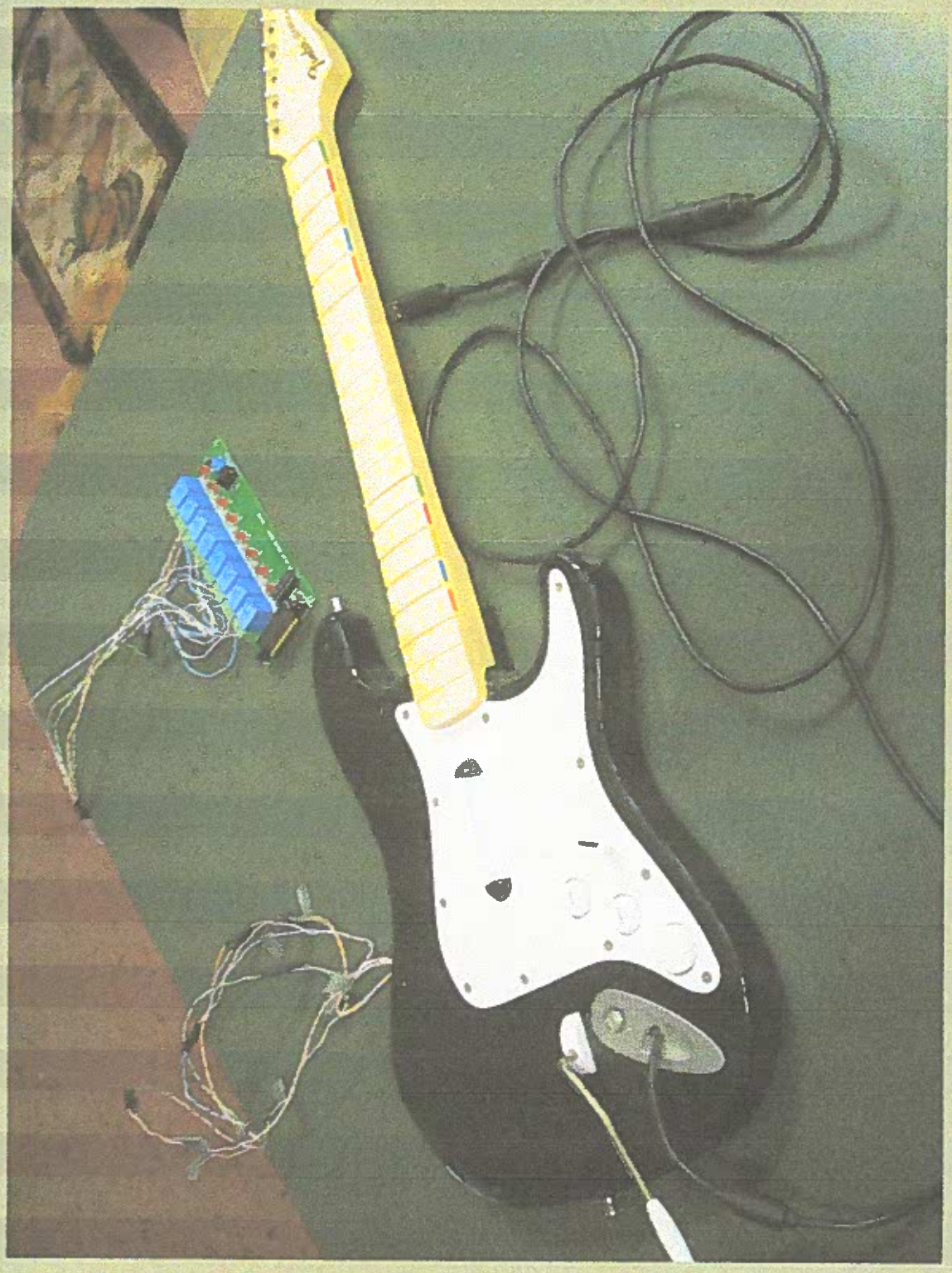


Guitar



Relay Board





How the Relay Board Works

- Meant for windows 95 or earlier
- User Port
- NC and NO
- Output hexadecimal integers to operate
  - 0x01 == 0000 0001 = relay one on
  - 0x03 == 0000 0011 = relays one and two on

How to play what notes and when

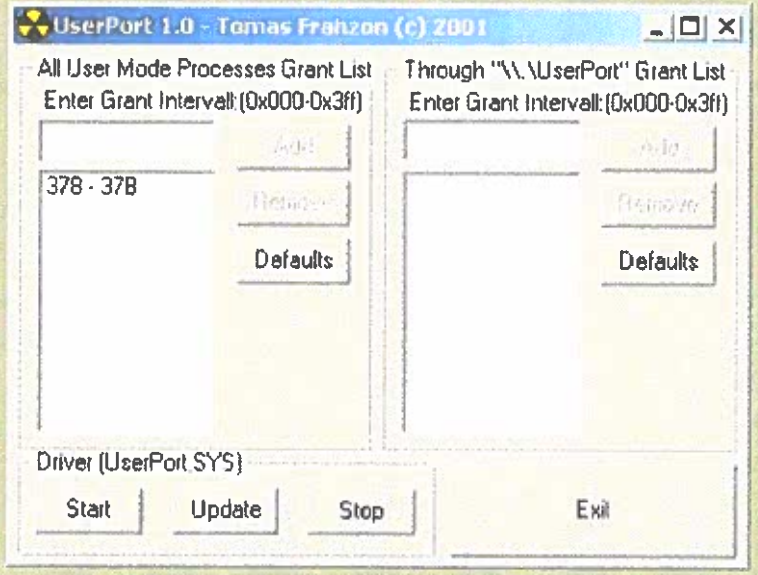
- PS2 version of Rockband
- Extr25
- Midi files
- Feedback
- Chart files

How to play what notes and when

- Chart files
  - Resolution
    - How many beats per tick
  - Sync track
    - 3840 = B 140900
  - Expert Single
    - 4080 = N 3 0
    - 4080 = N 4 0
- (Next Tick – Current tick)/resolution * 60/beats per minute
- Perl script

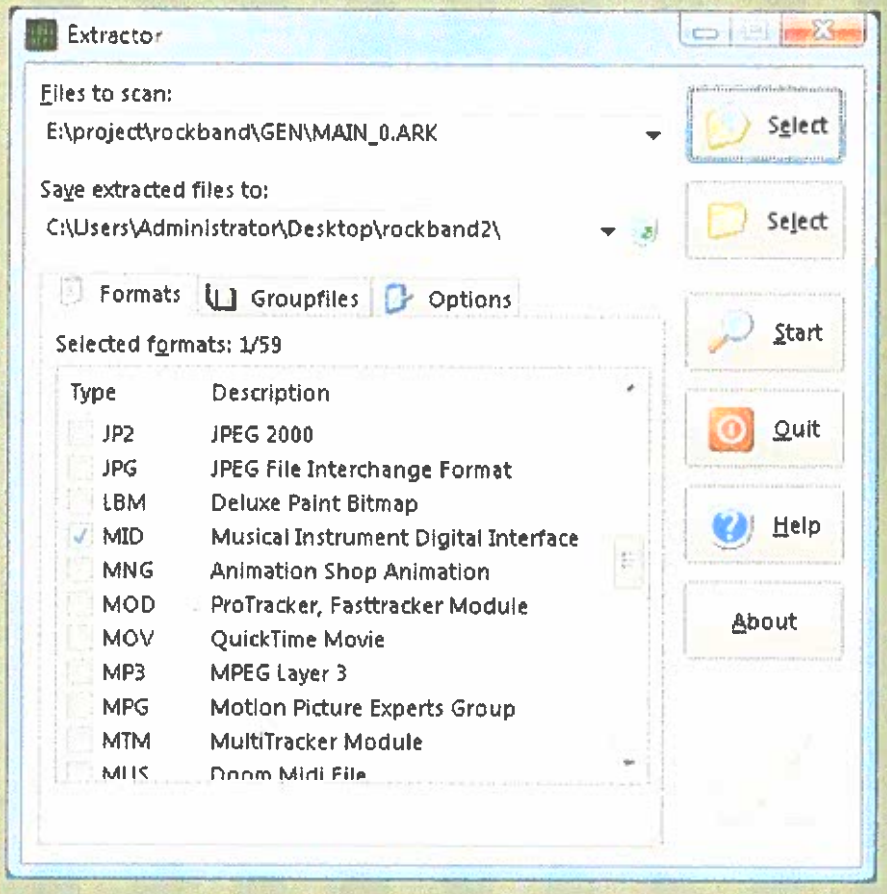
How to use

- Open user port and open the ports you need
  - I used LPT1 which is 0x378-0x37b



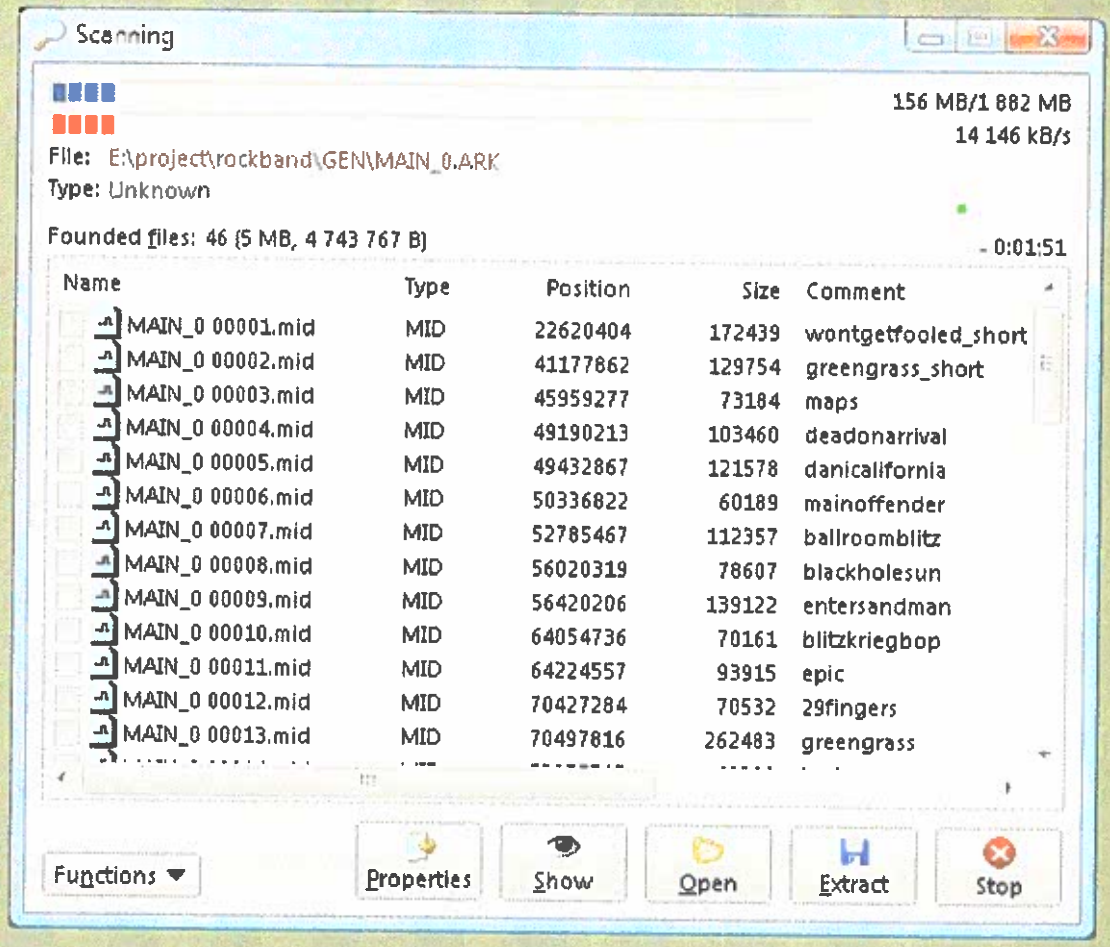
How to use

- Open up Extr25
  - Select MIDI
  - Navigate
    Files to scan
    to MAIN_0.ark
  - Set output directory
  - Click start



How to use

- Select all files
- Click extract

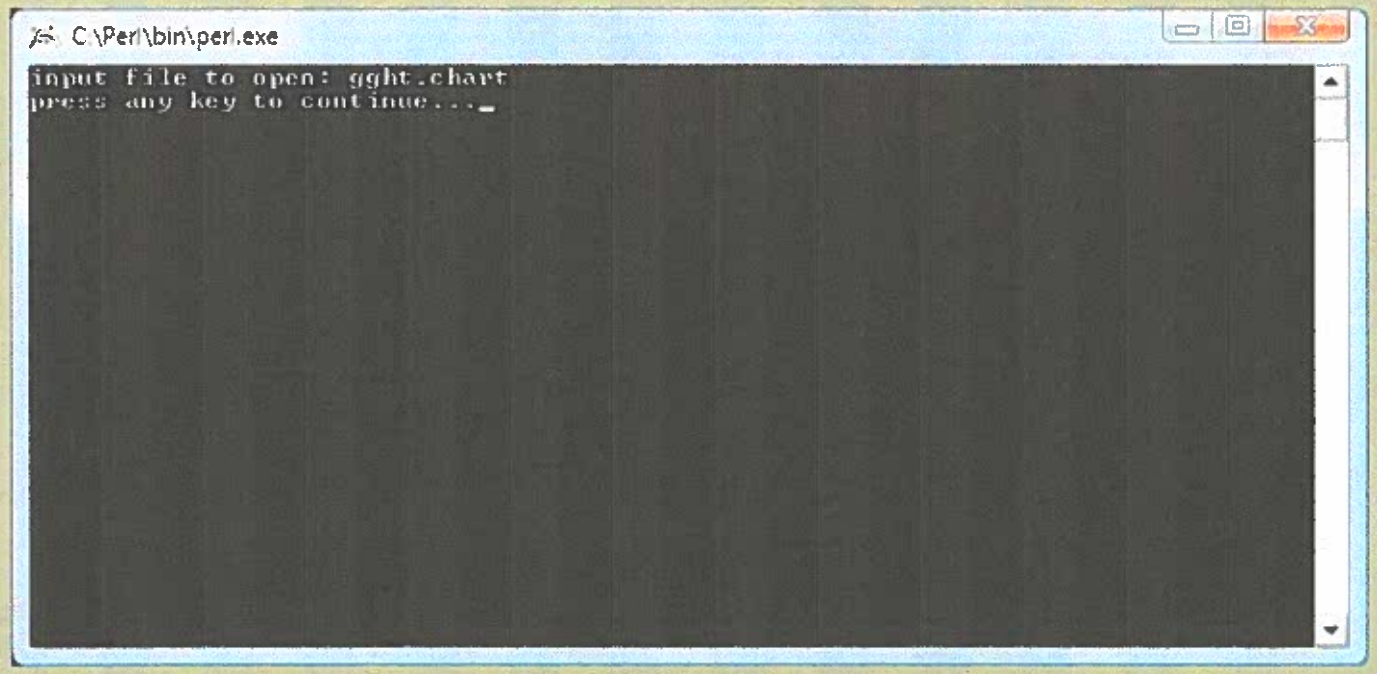


How to use

- Press escape to bring up menu
- Click load chart
- Find the song you want to load
- Save chart

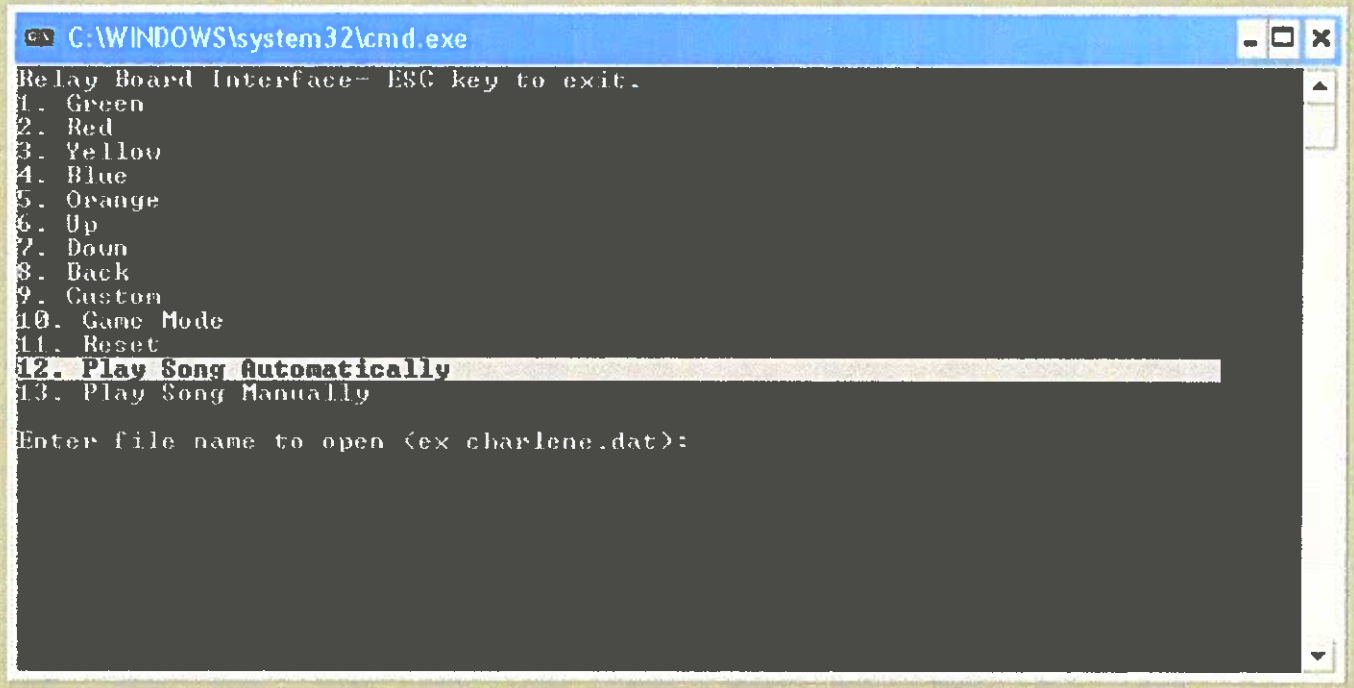
How to use

- Run chart2dat.pl
- Enter in the chart file you saved
- Press enter



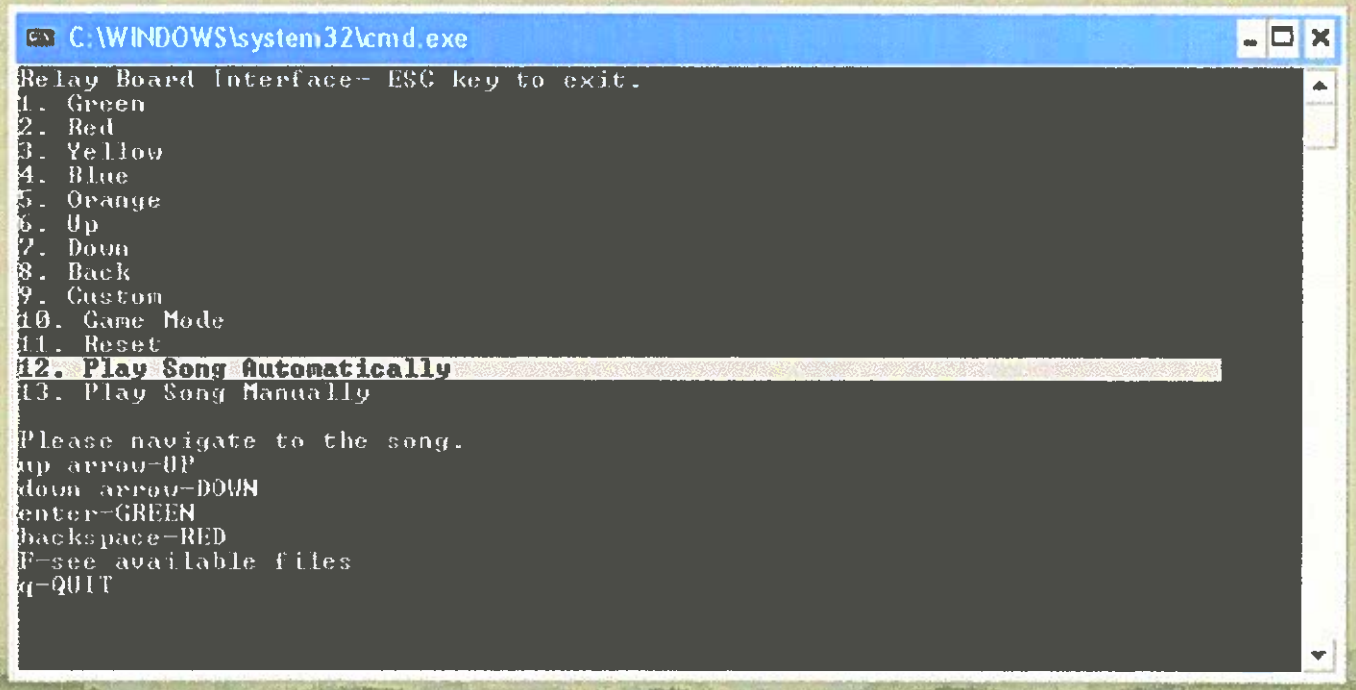
How to use

- Move the file to the directory with the main program
- Select play song automatically/manually depending on what you want to do



How to use

- Enter the song name
- Navigate to the song
- Once you are at the song and have selected the difficulty press Q



How to use

- When the first note of the song goes by, press any key to start the program playing the song.